

SYSTEMS & AI FULL-STACK ENGINEER RESUME

BUAA(北京航空航天大学) 软件学院

소프트웨어공학 · 2026.07 졸업 예정

왜 틀렸는지 끝까지 추적하는 개발자

Systems & AI Full-Stack Engineer · 한·중·영 트라이링귄

MIPS 마이크로커널을 Lab 0-6 전 단계 직접 구현하고, ECG 진단 AI 모델로 **Macro AUC 92.52%**를 달성한 **Systems & AI Full-Stack Engineer**입니다. 커널 IPC·컴파일러 파서·Spring Boot 백엔드·FastAPI 추론 서버를 한 사람의 손으로 만들어보며, **증상이 발생한 지점과 실제 원인은 항상 같은 곳에 있지 않다**는 원칙을 일하는 방식으로 만들었습니다. 실행 순서·상태 전이·API 규약·배포 전략 어느 계층이든 원인이 있는 곳까지 들어가 오류를 구조로 차단하는 백엔드·시스템 엔지니어가 되고자 합니다.

핵심 지표

92.52% ECG 진단 Macro AUC (F1 74.52%)

TPS 66.41 AWS 20명 동시 부하 처리

12배 Elasticsearch p95 응답 개선

9년 베이징 체류 · 한·중·영 트라이링귄

기술 스택

SYSTEMS

C / C++ · OS Kernel · Compiler · Virtual Memory · IPC

BACKEND

Java · Spring Boot 3 · PostgreSQL · Redis · Elasticsearch · MinIO

AI / APP

FastAPI · PyTorch · CNN-CBAM-GRU · React Native · Vue 3 + TS

INFRA

Docker Compose · AWS EC2 · GitHub Actions · JWT · Flyway

저는 이런 개발자입니다

- **구조로 강제하는 안정성:** API 응답 규약·예외 처리·배포 전략을 사람의 주의가 아닌 코드 계층에서 강제해, 휴먼 에러가 들어올 자리를 없앱니다.
- **로우레벨부터 서비스 레벨까지:** 커널 IPC, 컴파일러 파서, Spring Boot API, FastAPI AI 추론 서버를 한 사람의 손으로 만들어 본 경험으로, 추상화 아래에서 문제가 어떻게 발생하는지 압니다.
- **데이터로 합의하는 협업:** Vue vs React, 화웨이 클라우드 vs AWS 같은 팀 내 이견을 더미 10만 건 벤치마크와 RFC 문서로 정리해 합의 속도를 끌어올렸습니다.

핵심 역량

장애를 구조로 차단하는 설계

락 없는 IPC 레이스 컨디션, API 응답 규약 불일치, 저사양 EC2 OOM, AI 오류의 HTTP 200 포장 — 반복 가능한 장애를 실행 순서 재배치·공통 래핑 계층·배포 전략 전환·레이어 경계 재매핑으로 원인 자체를 제거했습니다.

비정상 상태에서도 무너지지 않는 구현

Fault-Tolerant 파서, responseBodyAdvice 기반 전역 응답 래핑, Cache-First + 도메인 이벤트 무효화를 설계하며, 비정상 입력과 중간 상태에서도 시스템이 일관성을 잃지 않도록 만들었습니다.

제한된 자원에서 운영하기

AWS 프리티어 t2.micro 위에서 AI 컨테이너와 백엔드를 분리 운영하고, Pull-only 배포·mem_limit 컨테이너 프로파일 분리로 서비스 가용성을 유지했습니다. 장애 진단은 docker logs → ps -a → journalctl → dmesg 4단 절차로 좁힙니다.

학력 · 어학 · 병역

학력	BUAA 软件学院 · SW 전국 1위 A+ 2020.09 – 2026.07 졸업 예정
고교	北京汇文中学 · 2017 – 2020 중국 체류 약 9년
중국어	TSC Level 6 · 비즈니스 커뮤니케이션
영어	OPIc IH · 기술 문서 독해 및 협업
병역	2023년 만기 전역 (병장)

일하는 방식

- 문제가 생기면 임시 수정으로 덮지 않고, 로그와 재현 조건을 먼저 고정한 뒤 원인 가설을 좁혀 갑니다.
- 팀의 합의가 필요한 사안은 RFC와 벤치마크 수치를 함께 제시해 결정 비용과 회의 시간을 줄입니다.
- 구현이 끝난 뒤에도 성능 수치·예외 케이스·배포 가능성을 점검해 운영할 수 있는 상태로 마무리합니다.
- 취미는 실내 클라이밍. 막힌 루트를 분석해 다시 도전하는 과정이 디버깅과 닮아 있다고 생각합니다.

직무경험 기술서

Projects 01 / 02 · Systems & Compiler

01 MIPS 마이크로커널 운영체제

2024.03 – 2024.07 · 1인 개발

OS Kernel 전체 구현 · 7단계 커널 완주 · Lock-free IPC race condition 해결

GXEmul MIPS 시뮬레이터 위에서 부팅 · 가상 메모리 · 프로세스 · 시스템콜 · IPC · 마이크로커널 FS까지 Lab 0-6 전 단계를 단독으로 완주하며, 추상화 아래에서 동시성과 메모리가 어떻게 동작하는지 코드로 확인했습니다.

구현 2단계 페이지 테이블과 TLB Fast Refill 핸들러를 어셈블리로 최적화하고, Copy-on-Write 기반 `fork()` 를 구현해 프로세스 생성 시 메모리 오버헤드를 낮췄습니다.

- 사용자 공간에서 동작하는 FS 서버 프로세스와 공유 페이지 기반 Pipe IPC를 구현해 커널과 파일 시스템을 물리적으로 분리했습니다.

문제 Pipe IPC 구현 중 스케줄링 타이밍에 따라 열린 파이프를 닫힌 것으로 오인하는 동시성 문제가 발생했습니다. 인터럽트 비활성화로 임시 봉합하는 대신 근본 원인인 **매핑 순서 의존성**까지 내려가 분석했습니다.

해결 `close()` 와 `dup()` 호출 시 페이지 map/unmap 순서를 재배치해 `pageref(fd) ≤ pageref(pipe)` 불변식이 항상 유지되도록 설계했고, 락 없이도 안정적인 동시성 제어를 구현했습니다.

기여 부팅부터 셸까지 7단계 전 과정을 1인으로 설계·구현했고, 특히 코드 실행 순서 재배치만으로 불변식을 강제하는 구조적 해결책을 도출했습니다.

C · MIPS Assembly · GXEmul Simulator · Microkernel FS · Pipe IPC · Virtual Memory

02 SysY 언어 컴파일러 프론트엔드 & VM

2024.10 – 2025.01 · 1인 개발

Lexer / Parser / Symbol Table / P-Code VM 전 파이프라인 구축 · a~m 전 에러 범주 처리 · Fault-Tolerant 파서

SysY 언어의 Lexer · 재귀 하향 Parser · Symbol Table · P-Code 생성기 · 스택 머신 VM까지 컴파일러 전 파이프라인을 C++17로 단독 구현했고, 구문 오류가 발생해도 파싱을 중단하지 않는 Fault-Tolerant 파서로 a~m 13개 에러 범주를 모두 처리했습니다.

구현 `unordered_map` 기반 키워드 판별과 `peek()` 메커니즘으로 `<=` · `==` · `&&` 등 다중 문자 연산자와 이스케이프 시퀀스 포함 문자열 상수를 일관되게 토큰화했습니다.

- CompUnit 선행 탐색과 `LVal = Exp vs [Exp]` 모호성 판별을 갖춘 재귀 하향 파서를 설계하고, 연산자 우선순위 기반 계층적 식 파싱을 구현했습니다.
- 스코프 ID와 스택 기반 심볼 테이블로 변수 새도잉, 함수 시그니처 불일치, 상수 재대입, 미선언 식별자 등 의미 오류를 컴파일 타임에 검출했습니다.
- 산술·논리·분기·반복·함수 호출·배열 인덱싱·I/O를 위한 P-Code 명령어 셋을 직접 설계하고, 이를 실행하는 스택 머신 인터프리터까지 구현했습니다.

문제 세미콜론 누락 케이스를 처리하려고 에러 함수를 추가하면, 기존에 통과하던 정상 파싱까지 깨지는 현상이 반복됐습니다. 단순한 패치로 덮는 대신 원인을 끝까지 추적해보니, `parseErrorI/J/K` 가 검사 함수가 아니라 `AdvanceToNextToken()` 으로 `mCurpos` 를 직접 변경하는 토큰 소비 함수였고, 호출 순서가 조금만 바뀌어도 후속 구문이 엉뚱한 토큰에서 시작하는 구조적 결함이 있었습니다.

- 해결** 에러 함수들을 **토큰 소비형(I/J/K)**, **look-ahead형(E)**, **심볼 등록형(B)**으로 먼저 분류했습니다. 그 위에서 에러를 파싱 중단 사유가 아닌 **문맥 복귀 신호**로 재정의하고, 새 복구 로직은 현재 위치를 저장한 뒤 필요한 토큰만 미리 확인하고 원위치로 되돌리도록 정리했으며, 누락 토큰은 직전 의미 단위 (`mLastNonT`) 기준으로 기록하도록 재설계했습니다.
- 학습** 왜 틀렸는지 끝까지 추적하는 과정이 제 개발 방식임을 이 프로젝트에서 확신했습니다. **상태를 바꾸는 함수와 읽기만 하는 함수를 구분**해야 한다는 원칙, 그리고 견고한 시스템은 정상 입력뿐 아니라 비정상 입력에서도 상태를 잃지 않아야 한다는 원칙을 이후 백엔드 API 예외 처리 설계에도 그대로 적용했습니다.

C++17 · CMake · Lexer / Parser · P-Code VM · Symbol Table · Error Recovery

직무경험 기술서

Projects 03 / 04 · Backend & AI Platform

03 학술 성과 공유 플랫폼

2025.03 – 2025.08 · 백엔드 전담

5인 다국적 팀 · 다중 인프라 통합 백엔드 · API 파싱 오류 0건 · ES p95 12배 개선

한국인 1명, 중국인 3명, 교환학생 1명으로 구성된 다국적 팀에서 PostgreSQL · Elasticsearch · Redis · MinIO · Vector Service를 통합한 학술 성과 검색·분석 백엔드를 전담했습니다.

- 구현**
- 팀원 간 OS와 DB 버전 파편화로 인한 인프라 기동 실패를 줄이기 위해 Docker Compose 기반 통합 개발 환경을 구축하고 README와 RFC 문서화를 주도했습니다.
 - 검색 성능 요구에 맞춰 PostgreSQL과 Elasticsearch를 함께 운용하고, Redis와 MinIO를 연결해 데이터 조회와 파일 저장 흐름을 분리했습니다.
- 문제**
- 개발자마다 API 응답 구조가 달라 프론트엔드에서 파싱 오류가 반복됐습니다. 문서화와 구두 합의만으로는 런칭 기한이 가까워질수록 휴먼 에러가 재발했고, 사람의 주의에 의존하는 한 끝나지 않는 문제임을 확인했습니다.
- 해결**
- Spring의 `ResponseBodyAdvice` 를 구현해 컨트롤러 반환값을 항상 `{code, message, data}` 구조로 자동 래핑했습니다. 규약 준수를 사람에게 맡기지 않고 아키텍처 계층에서 강제해 API 파싱 오류를 0건으로 수렴시켰습니다.
- 협업**
- Vue vs React, Huawei Cloud vs AWS 선택 이견을 더미 데이터 10만 건 기반 벤치마크(LIKE vs Elasticsearch 응답 시간)로 비교해, 감정이 아닌 수치로 합의를 이끌어냈습니다. 결론은 **Vue 3 + TypeScript**로 통일.
- 성능**
- 검색 p95가 SLA를 반복 초과해 `_explain.profile` 로 추적, 정확 매칭만 필요한 필드가 `text` 로 매핑돼 불필요한 분석기가 돌고 있었습니다. 새 인덱스 + `_reindex` + **index alias 무중단 교체**로 **p95 약 2,442ms → 200ms (12배) 개선**.

Spring Boot 3 (Java 21) · PostgreSQL 18 · Elasticsearch · Redis · MinIO · Vue 3 + TS · Docker Compose

04 CareLink — 실시간 노인 심전도(ECG) 진단 플랫폼

2025.10 – 2026.03 · 1인 개발 / 졸업논문

3-Tier 단독 풀스택 · CNN-CBAM-GRU 0.65M params · Macro AUC 92.52% / F1 74.52% · TPS 66.41 · 졸업 논문

환자와 보호자를 연결하는 모바일 앱과 실시간 ECG AI 진단 기능을 React Native · Spring Boot · FastAPI 3-Tier로 분리해, 1GB RAM EC2 위에서도 운영 가능한 구조로 단독 완성했습니다.

- 구현**
- PTB-XL 21,837건 · 12-lead 500Hz 데이터로 진폭 특성 36차원(Amplitude Feature Injection)을 설계해 **CNN-CBAM-GRU(0.65M params)**에 주입, **Macro AUC 92.52% / F1 74.52% / HYP F1 56.28%** 달성. 손실은 `AsymmetricLoss + pos_weight` 로 클래스 불균형을 보정했습니다.
 - 비즈니스 로직(Spring Boot)과 AI 추론(FastAPI)을 분리하고, JWT·자동 로그아웃·AsyncStorage **Cache-First** 전략으로 Home 첫 표시 0.29ms, 추론 P95 2,698ms 환경에서도 UX를 확보했습니다.
- 문제**
- FastAPI 추론 서버가 내부 오류를 JSON 필드에 담아 **HTTP 200으로 반환**해, 앱과 백엔드가 응답 코드만 보고 "성공" 처리하는 구조였습니다. 추론 실패가 사용자에게 "분석 완료"로 표시되는, 응답 코드 자체가 사실을 가리는 상황이었습니다.

- 해결** 오류 경계를 **레이어 진입 시점**에 정의하기로 하고, 백엔드에서 AI 응답을 검증해 추론 실패는 **502**, **타임아웃은 504**로 분리 매핑했습니다. 앱은 오류 유형별 복구 UI로 분기. 별도 OOM 이슈는 Pull-only 배포 + `mem_limit` + AI 컨테이너 `profile` 분리로 자원을 격리했고, 장애 진단은 `docker logs` → `ps -a` → `journalctl` → `dmesg` 순서로 좁혔습니다.
- 기여** 프론트·백엔드·AI 추론 3개 레이어의 설계·구현·배포를 1인 단독으로 수행. 모델 아키텍처 선정, 피처 설계, 손실 함수 조정, 추론 서버 배포까지 데이터 파이프라인 전체를 한 사람의 손으로 정리했습니다.

React Native · Spring Boot 3 (Java 21) · FastAPI · PyTorch · PostgreSQL 16 · AWS EC2 · Docker Compose · GitHub Actions